Conflict-Based Steiner Search for Multi-Agent Combinatorial Path Finding

Zhongqiang Ren, Sivakumar Rathinam, and Howie Choset

Boxiang Fu, Mehul Menon, TzuYi Yang

Carnegie Mellon University

Paper to Cover

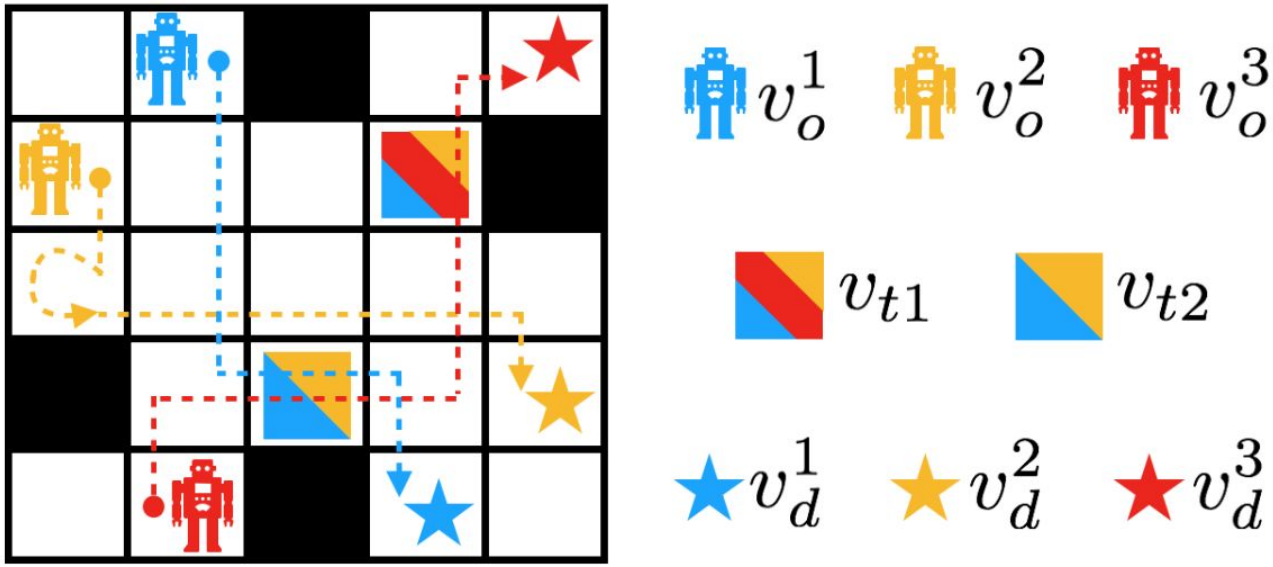
- **Title**: Conflict-Based Steiner Search for Multi-Agent Combinatorial Path Finding
- **Authors**: Zhongqiang Ren, Sivakumar Rathinam, Howie Choset
- **Affiliation**: Carnegie Mellon University, Texas A&M University
- **Conference**: Robotics: Science and Systems
- **Year**: 2022
- **Repository**: https://github.com/rap-lab-org/public_pymcpf
- **Website**: https://www.roboticsproceedings.org/rss18/p058.html

TLDR Summary

- Proposes an algorithm called Conflict-Based Steiner Search (CBSS) that combines Conflict-Based Search (CBS) and Multiple Traveling Salesman Problem (mTSP) to jointly assign intermediate targets and plan collision-free paths for multiple agents.
- Solves the problem of Multi-Agent Combinatorial Path Finding (MCPF), where robots must both determine who visits which intermediate targets and in what order while avoiding collisions.

Overview

- **MCPF (Problem)**: Multiple robots with starting locations and destinations. Must also visit a collection of intermediate waypoints with assignment constraints.



Overview

- **CBSS (Solution)**: Interleaves mTSP and CBS by alternating between
  - Generating new target sequences for the agents (which robot should visit each intermediate waypoint)
  - Finding conflict-free paths for agents given the target sequences (how the robots should travel given their intermediate waypoints)
- Algorithm does this by generating a search forest (a list of search trees). Each search tree is a particular target sequence (with some cost constraints). We search the tree for a conflict-free path for all agents. If this fails or costs grow large, we create a new search tree with another ordering of target sequences.
- Resulting algorithm is both **complete** and **optimal**. Optimality can be relaxed to bounded suboptimal in exchange for lower compute time.

Related Work

- **Multi-Agent Path Finding (MAPF)**: Finding paths for multiple agents avoiding collisions. Joint-State A*, CBS, PBS, PIBT, LaCAM*, Assignment 1.
- **Travelling Salesman Problem (TSP)**: Finding shortest path (tour) for a single agent. Integer Programming, LK/LKH (Heuristics), etc.
- **Multiple Travelling Salesman Problem (mTSP)**: Extends TSP to multiple agents. Agent-agent collisions not considered.
- **Target Assignment & Sequencing**: How to allocate a set of tasks (or intermediate waypoints) to different agents.

Problem Formulation: Notation

- **Definition**:
  - **Agents**: 1, 2, … N
  - **Workspace (Map)**: Graph $G = (V, E)$. Vertex set V represents the possible locations for agents. Edge set $E = V \times V$ denotes legal paths between vertices.
  - **Cost**: Travel along an edge incurs $\text{cost}(e) \in (0, \infty)$, where e is an edge.
  - **Waypoints**: $V_t \subseteq V \setminus \{V_o \cup V_d\}$. $f_A(v)$ is the subset of agents eligible to visit v.
- **Problem**:
  - Each target $v \in V_t$ is visited at least once by some agent in $f_A(v)$.
  - The path for each agent starts at its initial vertex and terminates at its destination vertex. Agents cannot collide with other agents on the path.
  - (If enforcing optimality) The sum of the cost of the paths is minimal.

Problem Formulation: Notation

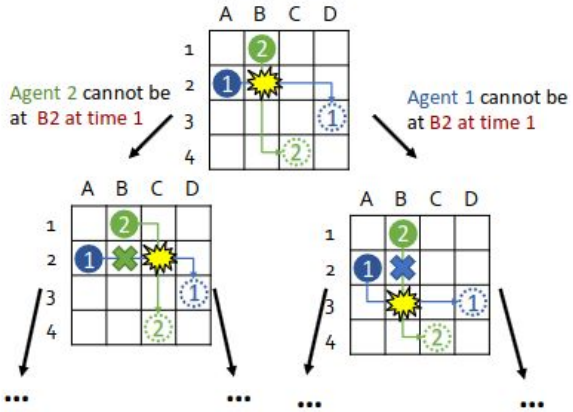
The sequence of targets is denoted by: $\gamma^i = \{v_o^i, u_1^i, u_2^i, .., u_l^i, v_d^i\}$

with initial and destination vertices of agent $i \in I$ denoted by v_o^i and $v_d^i \in V_d$, and the j-th intermediate target visited by agent i for j=1,..., ℓ is given by γ.

$\gamma = \{\gamma^i : i \in I\}$ denotes a collection of individual targets for all agents called a joint sequence.

The cost incurred in travelling between two nodes in γ is the same as the minimum cost for two nodes in G, and does not factor in collisions/conflicts.

A Quick Review of CBS



- A two level search algorithm for computing collision-free optimal solutions to MAPF problems.
- A joint path pi, with costs g, and a set of constraints ohm of the form (i,v,t) or (i,e,t) are computed for an agent i.
- A Search Tree is constructed
  - Initial paths are planned without conflicts
  - Every pair of paths is checked for a conflict
  - A new constraint set is generated for every conflict detected
  - New paths are planned with the new constraint sets

Carnegie Mellon University

Problem Formulation: The Objective of CBSS

CBSS constructs a search forest instead of a search tree

Each tree in the forest corresponds to a given joint sequence γ_j^*

The joint sequences are generated with monotonically increasing costs

$cost(\gamma_1^*) < cost(\gamma_2^*) < cost(\gamma_3^*) < \ldots$

The joint sequences are initially generated without considering conflicts

A low-level Planner incorporates conflicts like CBS

➔ It ensures that the cost of the generated plan does not exceed the next best Search Tree

This is guaranteed to find an optimal solution to the MCPF

CBSS Notation

- Let, $G_T = (V_T, E_T, C_T)$ denote a target graph
- The graph is undirected with the vertex set $V_T = V_o \cup V_t \cup V_d$ and Edge set E_T.
- The cost of each edge in E_T is denoted by C_T, a Cost Matrix of size $(2N + M) \times (2N + M)$.
- Each edge $(u, v) \in E_T$ represents a minimum cost path connecting (u, v) in graph G, ignoring conflicts

Generation of a New Root

In each iteration of the high-level search, for the node P_l:

The node P_ℓ with least g-value is popped from OPEN

Compare the cost of P_ℓ with a threshold

If no node needs to be created, P_ℓ is returned

Otherwise, a new root node is generated using the K-best sequences method.

➔ The low-level plan computes a new joint path γ_r^* for the root node.

K-best Joint Sequences: Restricted TSP/rTSP Formulation

The objective: Convert mTSP into equivalent TSP on an augmented graph, following which find the K cheapest solutions after applying the partition-based methods.

Given a directed graph, $G' = (V', E', C')$, let $I_e, O_e \subseteq E'$, denote two disjoint subsets of edges in G', an rTSP finds a minimum cost tour τ^* such that $I_e \subseteq \tau^* \subseteq E' \backslash O_e$.

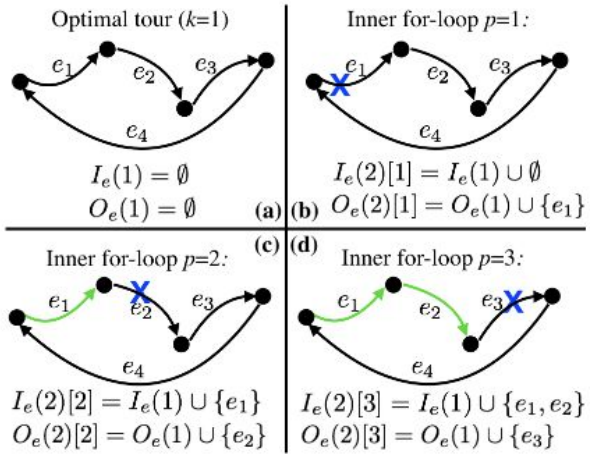
This done by temporarily setting the costs of the edges in O_e to large positive values. A regular TSP solver is invoked to solve the rTSP then.

K-best Joint Sequences: Partitioning Scheme

- We then compute the K best TSP by iteratively partitioning the set of feasible tours while finding an optimal tour in each partitioned subset

- For the k-th while iteration:
  - The K-best solutions are added to K when k=K
  - Otherwise,
    - the edges in the tour τ_k^* continue to be indexed from 1 to ℓ.
    - An inner for loop over all edges to partition the set of remaining feasible tours iteratively to generate a corresponding rTSP

mTSP to TSP Transformation for Computing K-Best Sequences



1. Convert a target graph GT into a transformed graph GTF based on assignment constraints and edge costs

$$G_{TF} = (V_{TF}, E_{TF}, C_{TF})$$

2. Compute edge-specific costs for Type-1, Type-2, Type-3 and Type-4 edges

3. A minimum cost tour is computed on GTF with a TSP solver

4. The obtained optimal tour is converted into a joint sequence
   a. Breaking the tour into N segments by removing all zero-cost edges from the destination to the start
   b. Copies of the same targets are shortcutted
   c. Sum up cost of edges present in the joint sequence wrt values in the cost matrix of the target graph to get the cost of the resulting joint sequence

Carnegie Mellon University

CBSS Overview

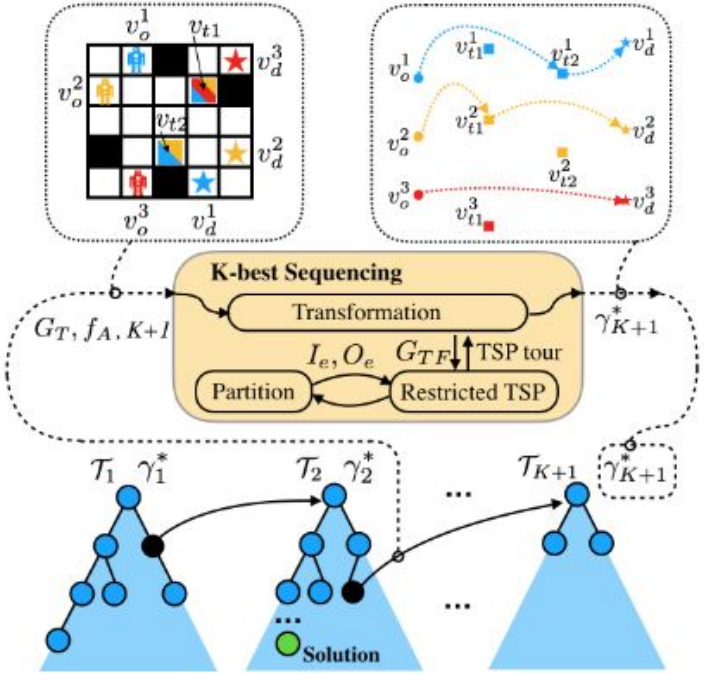


Fig. 2: A conceptual visualization of the CBSS framework.

Algorithm 1 Pseudocode for CBSS

Compute $G_T = (V_T, E_T, C_T)$
$\gamma_1^* \leftarrow$ *K-best-Sequencing*$(G_T, f_A, K = 1)$
$\Omega \leftarrow \emptyset$
$\pi, g \leftarrow$ *LowLevelPlan*(γ_1^*, Ω)
Add $P_{root,1} = (\pi, g, \Omega)$ into OPEN
while OPEN not empty **do**
  $P_l = (\pi_l, g_l, \Omega_l) \leftarrow$ OPEN.pop()
  $P_k = (\pi_k, g_k, \Omega_k) \leftarrow$ *CheckNewRoot*$(P_l$, OPEN$)$
  DetectConflict(π_k)
  if no conflict detected in π_k **then**
    return π_k
  $\Omega \leftarrow$ Split the detected conflict
  for all $\omega^i \in \Omega$ **do**
    $\Omega'_k = \Omega_k \cup \{\omega^i\}$
    $P'_k \leftarrow$ *LowLevelPlan*$(\gamma(P_k), \Omega'_k)$
    // In this LowLevelPlan, only agent i's path is planned.
    Add P'_k to OPEN
return failure

Analysis – Completeness

- **Finite Search Space:** For a feasible MCPF instance, CBSS is guaranteed to return a feasible solution because it explores a finite number of possible root nodes
- **Systematic Exploration:** Within each search tree, CBSS expands nodes with non-decreasing costs (best-first), and there are only a finite number of possible nodes within a specific cost bound.
- **Guaranteed Termination:** The algorithm will eventually find a conflict-free path if one exists by traversing the search forest systematically.

Analysis – Optimality

- **The Optimal Case (ε=0):**
  - **Best-First Order:** CBSS expands the node with the minimum g-value from the global open queue.
  - **Global Optimum:** Since root nodes (new trees) are generated in monotonically increasing order of cost, the first conflict-free node expanded must be the minimum cost among all un-expanded nodes.

Analysis – ε-Suboptimality

- **Bounded Proof:**
  - Let g* be the optimal cost and P* be the optimal search node
  - Let g be the cost of the solution P returned by CBSS.
  - let P_root* denote the root node of the tree that contains P*
  - P is returned, the root node P_root* is either (1) generated or (2) not generated

Analysis – ε-Suboptimality

For Case (1) – P_root* is generated:

Because CBSS expands nodes in a best-first manner, P is the first conflict-free node expanded.

g is the minimum cost among all un-expanded nodes in OPEN, meaning $g \leq g^*$

the bound holds: $g \leq g^* \leq (1+\varepsilon)g^*$

For Case (2) – P_root* is not generated:

CBSS will not expand any node P if its cost exceeds the threshold of the next potential root.

node is only expanded if: $g \leq (1+\epsilon)cost(\gamma^*_{r+1})$

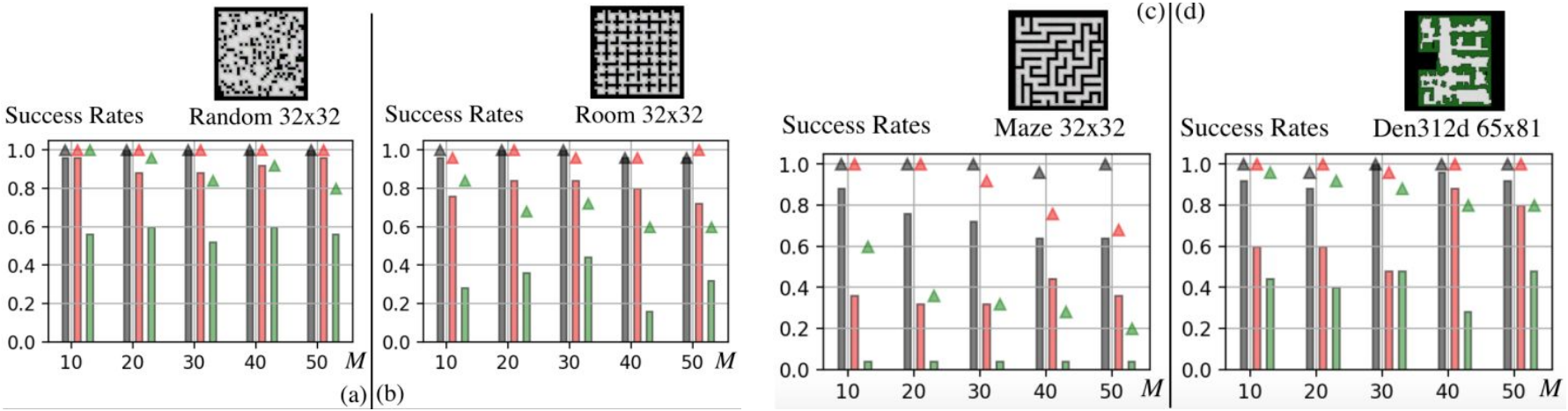
Since root costs are monotonically increasing: $g^* \geq cost(\gamma^*_{r'}) \geq cost(\gamma^*_{r+1}) \geq \frac{g}{(1+\epsilon)}$

In both case we get the result of $g \leq (1+\varepsilon)g^*$

Results – Success Rates

- **Time Constraint:** One-minute runtime limit
- **Hyperparameter:** ε set to zero for optimal solution
- **CBSS vs. MS*:** CBSS achieved up to 60% higher success rates than the baseline MS*

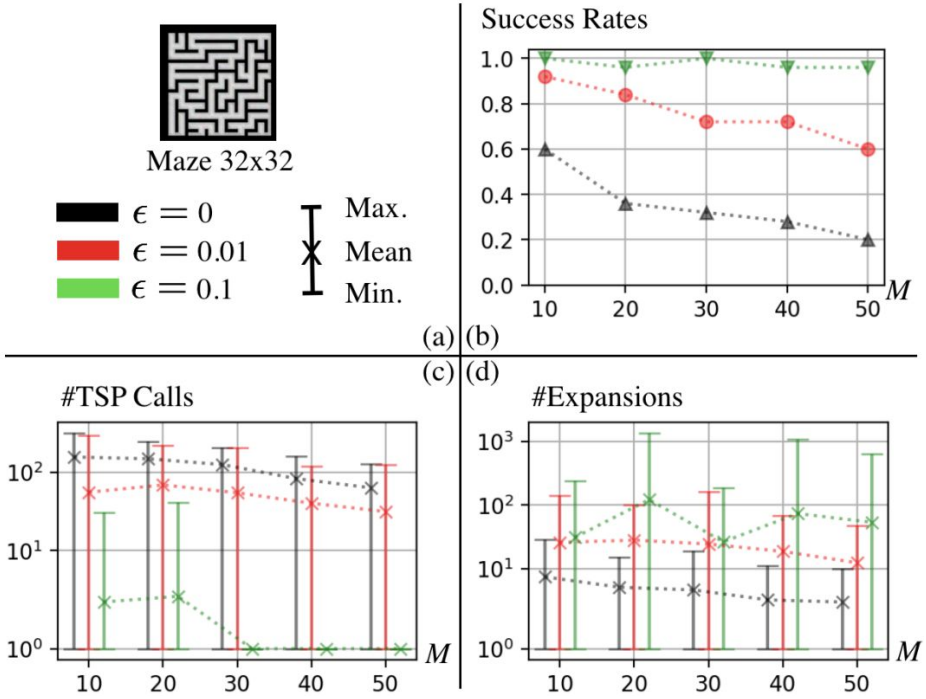
	N=5	N=10	N=20
MS* (baseline)	■	■	■
CBSS (this work)	▲	▲	▲



Carnegie Mellon University

Results – Impact of Sub-Optimality Bound

- One-minute runtime limit
- Larger ε tends to defer the generation of the next best joint sequence



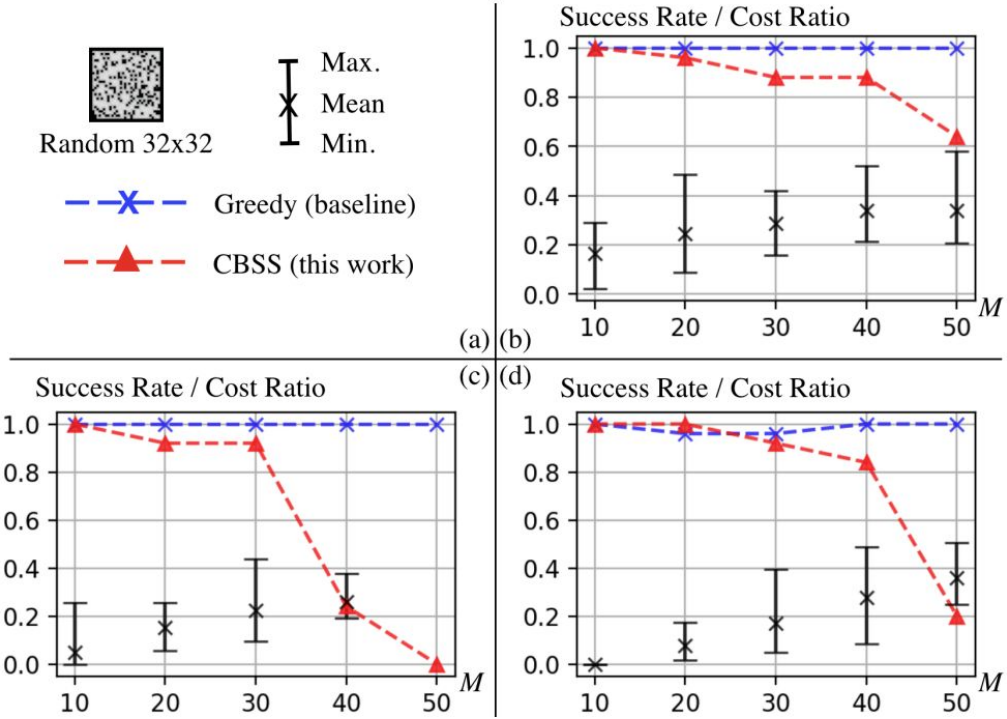
Results – CBSS vs. Greedy

- **Three Types of Cases**
  - **Unique Destinations:** targets are anonymous.
  - **Pre-assigned Targets:** agent has one pre-assigned target. Other targets and all destinations are anonymous.
  - **Mix Constraints:** Destination is assigned to a unique agent and a pre-assigned targets while the remaining targets are anonymous.
- **Setting:** ε set to 0.01 and 10 agent.
- **Time Constraint:** One-minute runtime limit

Results – CBSS vs. Greedy

$$\text{cost ratio} = \frac{cost(\pi_{Greedy}) - cost(\pi_{CBSS})}{cost(\pi_{CBSS})}$$

- The greedy method generally achieves higher success rates than CBSS.
- the greedy approach paths often being much more expensive than those found by CBSS.



Carnegie Mellon University

Conclusion

- **The CBSS Framework:** Proposed a search forest architecture that interleaves Conflict-Based Search (CBS) for collision avoidance with Traveling Salesman (TSP) algorithms for optimal sequencing.
- **Flexible Sub-Optimality:** Introduced the ε parameter, allowing users to trade a proven amount of path quality for faster computation and higher success rates.
- **Higher Success Rates:** Achieved up to 60% higher success rates than the MS* within a one-minute runtime limit.
- **Superior Cost-Effectiveness:** Produced paths that are shorter than greedy baselines, saving up to 50% in total path cost.

Thank You!